Project 4: Autostitching Photo Mosaics

An automatically-stitched mosaic of the Berkeley campus!

Part A.1: Shooting the Pictures

First, I captured a few sets of photographs of the same scene by rotating a camera while trying to keep its center of projection fixed. This results in photographs which can be transformed into one another via a projective transform.

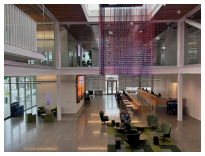
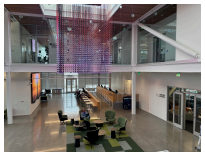



Image 1 of the Grimes Engineering Center

Image 2 of the Grimes Engineering Center

Image 3 of the Grimes Engineering Center

Image 1 of the Berkeley campus

Image 2 of the Berkeley campus

Image 3 of the Berkeley campus

Image 4 of the Berkeley campus

Image 5 of the Berkeley campus

Part A.2: Recover Homographies

Next, I needed to find the transformation matrix H which could be applied to one image in order to warp it onto a shared projective plane as the second image. In this case, the transformation matrix is a homography matrix H with dimensions (3, 3) and 8 degrees of freedom (the lower right element is a scaling factor that can be set to 1).

A correspondence is a pair of pixel points (p', p) in two images that represent the same point in the 3D world.

Given a point p' in image 1 and a point p in image 2, left-multiplying the homogeneous coordinates of p by H maps it to p'. Dividing the resulting 3D vector by its last component to normalize it into a homogeneous coordinate with depth 1 gives:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

$$= \begin{bmatrix} (ax + by + c) \\ (dx + ey + f) \\ (gx + hy + 1) \end{bmatrix}$$

$$= \begin{bmatrix} \frac{(ax+by+c)}{(gx+hy+1)} \\ \\ \frac{(dx+ey+f)}{(gx+hy+1)} \\ \\ 1 \end{bmatrix}$$

Looking at just the non-homogeneous coordinates:

$$x' \cdot (gx + hy + 1) = ax + by + c,$$
$$y' \cdot (gx + hy + 1) = dx + ey + f$$

Rewriting the equations to isolate x' and y' on the left-hand side:

$$x' = ax + by + c - x' \cdot gx - x' \cdot hy$$
$$y' = dx + ey + f - y' \cdot gx - y' \cdot hy$$

Therefore, pair of points (p', p) gives us 2 equations. Since H has 8 degrees of freedom, we need at least 4 pairs of points to have a determined system of equations. Then, we will be able to set up a linear system of

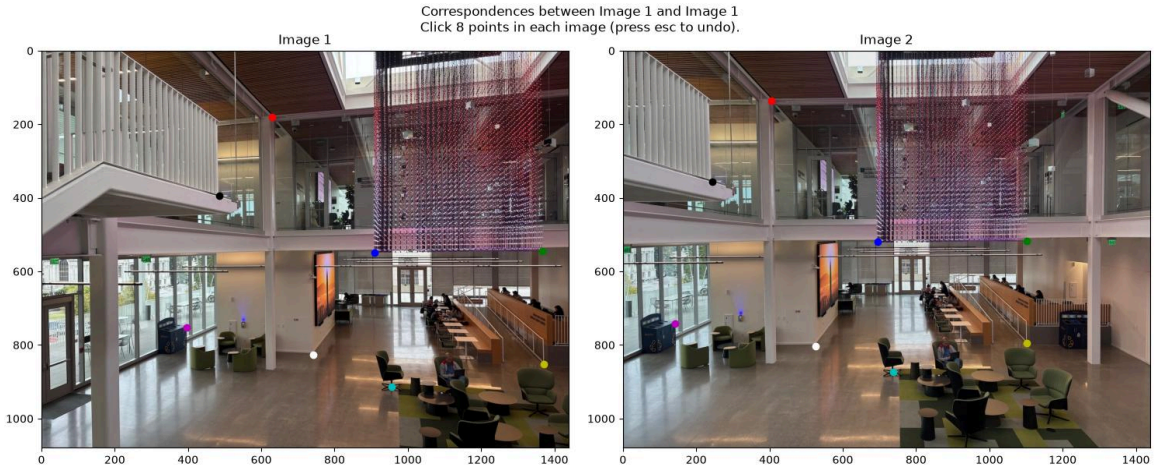
equations in the form of Ah=b, where h is a 8D vector holding the 8 entries of H.

$$\begin{bmatrix} x_1' \\ y_1' \\ x_2' \\ y_2' \\ \vdots \\ x_n' \\ y_n' \end{bmatrix} = \begin{bmatrix} x_1 & y_1 & 1 & 0 & 0 & 0 & -x_1' \cdot x_1 & -x_1' \cdot y_1 \\ 0 & 0 & 0 & x_1 & y_1 & 1 & -y_1' \cdot x_1 & -y_1' \cdot y_1 \\ x_2 & y_2 & 1 & 0 & 0 & 0 & -x_2' \cdot x_2 & -x_2' \cdot y_2 \\ 0 & 0 & 0 & x_2 & y_2 & 1 & -y_2' \cdot x_2 & -y_2' \cdot y_2 \\ & & & & \vdots & & & \\ x_n & y_n & 1 & 0 & 0 & 0 & -x_n' \cdot x_n & -x_n' \cdot y_n \\ 0 & 0 & 0 & x_n & y_n & 1 & -y_n' \cdot x_n & -y_n' \cdot y_n \end{bmatrix} \begin{bmatrix} a \\ b \\ c \\ d \\ e \\ f \\ g \\ h \end{bmatrix}$$

The solution h will be unstable and prone to noise with only 4 points because an error of just a couple of pixels can result in very big changes in the recoevered homography. Thus & it's useful to instead use more than 4 point pairs, creating an over-determined system of equations, then find an h via least-squares.

In part A, I found correspondences between 2 images by implementing my own interactive widget that shows two images side-by-side and lets users select points in each image.

This is an example of how my correspondence-selection and homography algorithms work! This example used 8 correspondence point pairs, resulting in a system of 16 equations.



Manually-selected correspondences between 2 images of the Grimes Engineering Center.

$$Ah = b$$

$$\begin{bmatrix} 9.09 \times 10^2 & 5.50 \times 10^2 & 1.00 & 0 & 0 & 0 & -6.32 \times 10^5 & -3.82 \times \\ 0 & 0 & 0 & 9.09 \times 10^2 & 5.50 \times 10^2 & 1.00 & -4.72 \times 10^5 & -2.85 \times \\ 1.37 \times 10^3 & 5.45 \times 10^2 & 1.00 & 0 & 0 & 0 & -1.51 \times 10^6 & -6.01 \times \\ 0 & 0 & 0 & 1.37 \times 10^3 & 5.45 \times 10^2 & 1.00 & -7.08 \times 10^5 & -2.82 \times \\ 6.30 \times 10^2 & 1.81 \times 10^2 & 1.00 & 0 & 0 & 0 & -2.55 \times 10^5 & -7.33 \times \\ 0 & 0 & 0 & 6.30 \times 10^2 & 1.81 \times 10^2 & 1.00 & -8.59 \times 10^4 & -2.47 \times \\ 9.56 \times 10^2 & 9.14 \times 10^2 & 1.00 & 0 & 0 & 0 & -7.05 \times 10^5 & -6.74 \times \\ 0 & 0 & 0 & 9.56 \times 10^2 & 9.14 \times 10^2 & 1.00 & -8.35 \times 10^5 & -7.99 \times \\ 3.96 \times 10^2 & 7.53 \times 10^2 & 1.00 & 0 & 0 & 0 & -5.64 \times 10^4 & -1.07 \times \\ 0 & 0 & 0 & 3.96 \times 10^2 & 7.53 \times 10^2 & 1.00 & -2.94 \times 10^5 & -5.59 \times \\ 1.37 \times 10^3 & 8.53 \times 10^2 & 1.00 & 0 & 0 & 0 & -1.51 \times 10^6 & -9.40 \times \\ 0 & 0 & 0 & 1.37 \times 10^3 & 8.53 \times 10^2 & 1.00 & -1.09 \times 10^6 & -6.78 \times \\ 4.84 \times 10^2 & 3.94 \times 10^2 & 1.00 & 0 & 0 & 0 & -1.18 \times 10^5 & -9.57 \times \\ 0 & 0 & 0 & 4.84 \times 10^2 & 3.94 \times 10^2 & 1.00 & -1.72 \times 10^5 & -1.40 \times \\ 7.42 \times 10^2 & 8.27 \times 10^2 & 1.00 & 0 & 0 & 0 & -3.89 \times 10^5 & -4.34 \times \\ 0 & 0 & 0 & 7.42 \times 10^2 & 8.27 \times 10^2 & 1.00 & -5.96 \times 10^5 & -6.65 \times \end{bmatrix}$$

Solving this system of equations via NumPy's linear algebra least-squares function gave this as the solution for h:

$$h = \begin{bmatrix} 1.35 \\ 9.35 \times 10^{-3} \\ -3.86 \times 10^{2} \\ 1.24 \times 10^{-1} \\ 1.21 \\ -1.40 \times 10^{2} \\ 2.37 \times 10^{-4} \\ 1.66 \times 10^{-5} \end{bmatrix}$$

Finally, reshaping the vector h into a 3x3 matrix gives the homography matrix H:

$$H = \begin{bmatrix} 1.35 & 9.35 \times 10^{-3} & -3.86 \times 10^{2} \\ 1.24 \times 10^{-1} & 1.21 & -1.40 \times 10^{2} \\ 2.37 \times 10^{-4} & 1.66 \times 10^{-5} & 1.00 \end{bmatrix}$$

Part A.3: Warp the Images

After recovering the homography, I warped one image onto another "reference" image's projective plane. I implemented this via inverse warping! First, I computed what the pixel positions of image 1's corners would be after warping. I used this as the bounding box to create an empty "canvas" of all zeros. Then, I iterated through all the pixel positions in the canvas. For each pixel position, I applied the inverse of the homography matrix to find which pixels in the source image it came from. Then, I "gathered" the values from the source pixels and set the output pixel to the gathered colors.

I implemented 2 interpolation methods from scratch:

Nearest neighbor interpolation: after inverting the output pixel coordinates, I rounded the resulting coordinates to find the nearest, valid source pixel position to gather a value from.

Bilinear interpolation: after inverting the output pixel coordinates, I computed the coordinates of the 4 neighboring pixels from the source image around the inverted coordinate, then used a weighted average of the neighbors' values in the output based on how close the inverted pixel coordinate is to each neighbor. To determine how much to weigh each neighbor's values, we can visualizethe neighbors as forming a 1 pixel by 1 pixel square. The inverted pixel coordinates fall somewhere inside this square (possibly on an edge of the square or on one of the neighbors themselves). The inverted pixel splits the 1x1 square into 4 sub-rectangles (or fewer than 4 sub-rectangles if the inverted pixel lies on an outer edge/corner). Then, each neighbor should be assigned a weight proportional to the area of the sub-rectangle diagonal from it. For example, the neighbor above and to the left of the inverted pixel should be weighted by an amount proportional to the area of the bottom-right sub-rectangle.

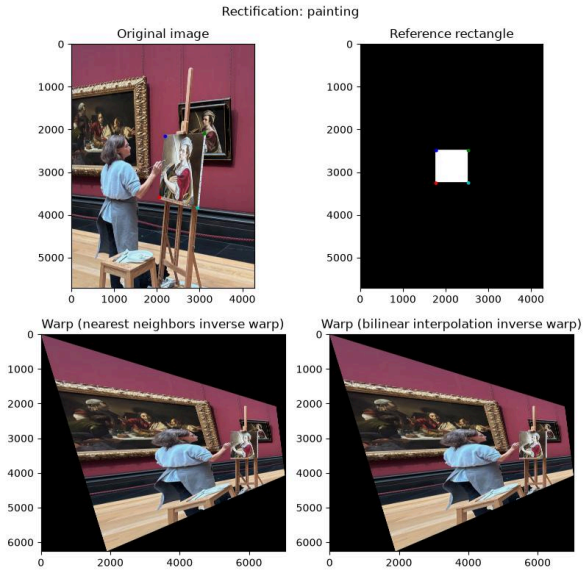
For both interpolation methods, if the pixel in the warped output canvas was inverted to a coordinate that would be out-of-bounds in the source image, I treated that source pixel as having a value or weight of 0.

These two interpolation methods trade-off speed and quality. While nearest neighbors is algorithmically simpler and usually faster because it only requires "gathering" from at most one source pixel for every pixel in the output, bilinear interpolation can reduce image artifacts in the output since it more smoothly "blends"/combines colors from the source image in the output.

To test my homography and interpolation code, I applied them to pictures

depicting known rectangular objects from non-front-on angles. This is known as "rectification": projecting images onto a common image plane (in this case, projecting images onto an plane such that the skewed rectangular objects look like rectangle again). Here are the results of rectifying a painting (*of a painting* in The National Gallery located in London!) and a fresco in Pompeii. I knew that in real-life, these paintings and frescoes were squares/rectangles with right angles and parallel edges. But because the images were taken from an angle, the paintings/frescoes look like skewed quadrilaterals. Rectifying these images involves pairing these images with a placeholder image of a normal rectangle to represent a projective plane that views the paintings/frescoes from front-on, computing a homography, and interpolating the colors from the original image to "unskew" the subject.

After rectification, the selected subject will now appear to have right angles and parallel lines!

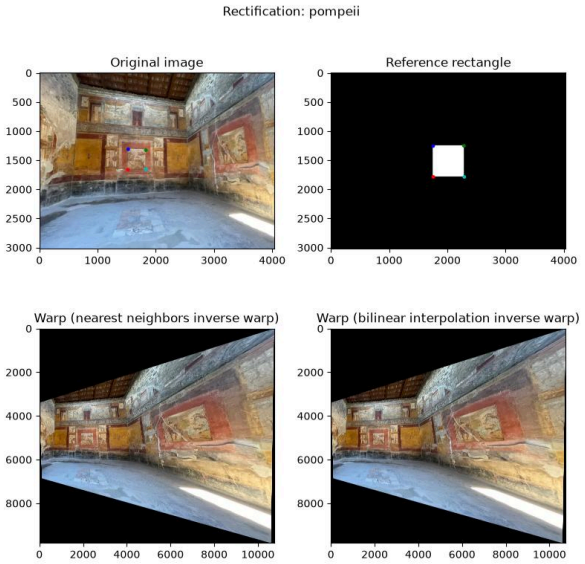


Rectifying a live painting of a painting. The top row shows the correspondences between corners of the painting and a reference square. The

bottom row compares the results with both nearest neighbors interpolation and bilinear interpolation.

painting.jpg's rectification using nearest neighbors interpolation.

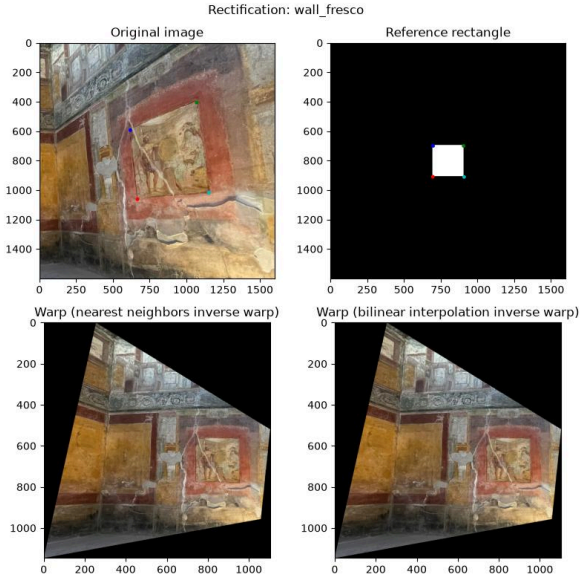
painting.jpg's rectification using bilinear interpolation.



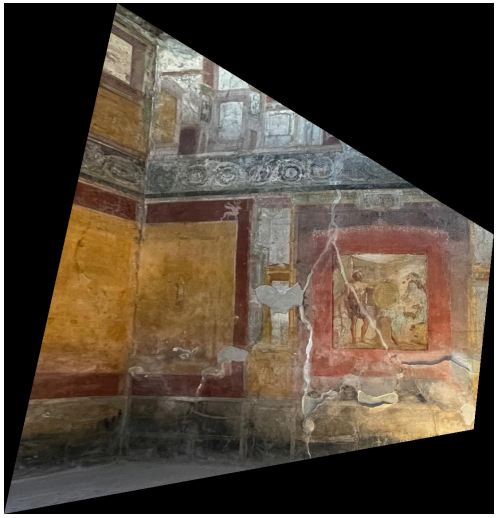
Rectifying a Pompeiian fresco.

pompeii.jpg's rectification using nearest neighbors interpolation.

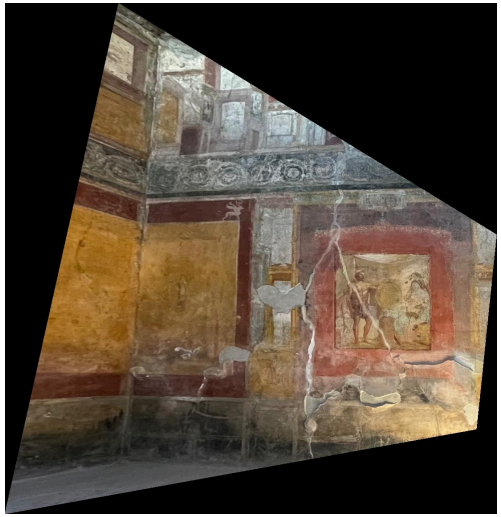
pompeii.jpg's rectification using bilinear interpolation.



Rectifying the another Pompeiian fresco! This time, the fresco on the right wall in the picture.



wall_fresco.jpg's rectification using nearest neighbors interpolation.



wall_fresco.jpg's rectification using bilinear interpolation.

Part A.4: Blend the Images into a Mosaic

With all these puzzle pieces in place, we can now stitch multiple overlapping images into an image mosaic! In my implementation, I treated the middle image as the center of the mosaic, then transformed all the other images onto this center image's projective plane. To avoid edge effects, I used multi-resolution blending with Laplacian and Gaussian stacks.

1. Find correspondences between adjacent images and compute Homographies that map adjacent images to each other.

2. Compose the transforms together to find a homography for each image that maps it onto the shared, common projective plane (AKA the projective plane that the center image uses).

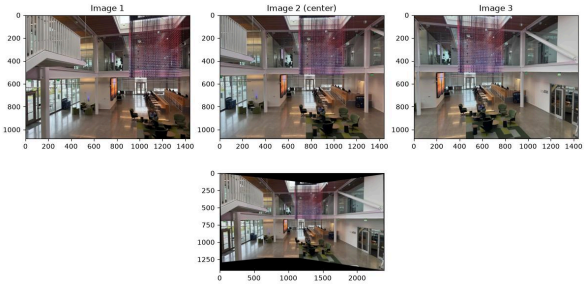
3. Compute the size that the shared canvas should have so that it fits all the images after they are tranformed. For each image, transform its 4

corners using the image-to-center homography. Then, find the minimum x, minimum y, maximum x, and maximum y coordinates across all the warped corners. The shared canvas needs to be able to fit the full range of x, y coordinates; thus the dimensions of the final mosaic canvas should equal (maximum x - minimum x + 1, maximum y - minimum y + 1) pixels.

4. Warp each image onto its own copy of the canvas using the image-to-center homography and inverse interpolation. Using the same sized canvas helps simplify the multi-resolution blend step. Also, create a mask for each iamge. The mask should have the same shape as the canvas and have 1's where the warped image is located, and 0's elsewhere.

5. Blend all the image canvases together using multi-resolution blending! This involves creating Laplacian stacks of all the canvases containing a single warped image and Guassian stacks of all the masks. Then, merge each level by weighting each canvas's Laplacian at that level by its mask's Gaussian at that level. Finally, collapse (add) the blends from all the levels together.

Here are 3 photo mosaics I created from pictures I took on campus. There are still some blending artifacts like ghosting and blurriness. I attribute these to the fact that I took these photos by hand so the camera's center of projection may have slightly shifted between photos even though I tried my best to keep it fixed, and due to the fact that I manually selected corresponding keypoints between adjacent photos but the homography transform can be sensitive to even small pixel differences. Overall however, I think the resulting photo mosaics present a very *picturesque* look at everyday life at Cal :)



Images from inside the Grimes Engineering Center

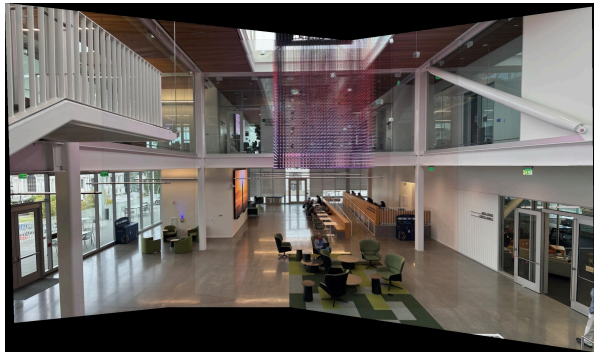
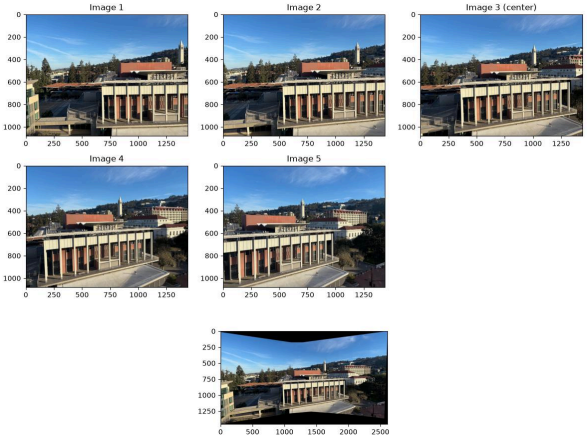
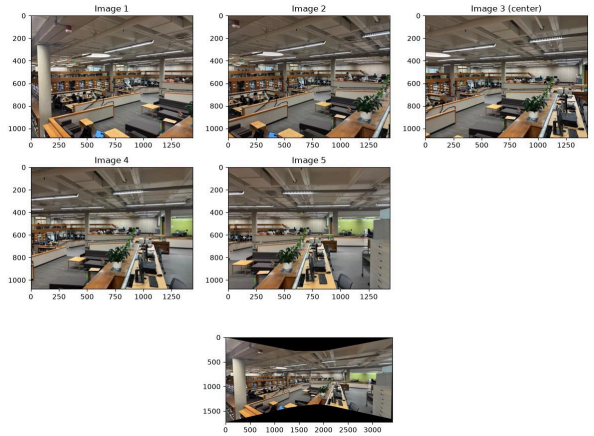


Photo mosaic of the Grimes Engineering Center.



Images of the Berkeley campus seen across from the Student Union.

Photo mosaic of Berkeley campus.



Images from inside the Kresge Library.

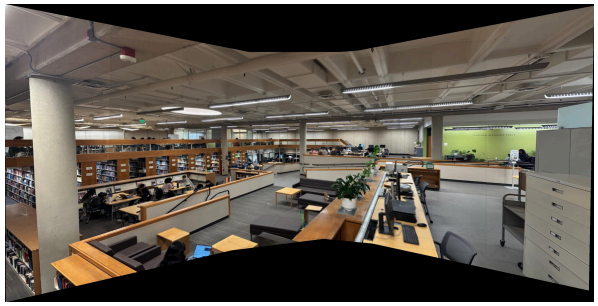


Photo mosaic of Kresge Library.

Part B.1: Harris Corner Detection

To automatically stitch together images into a mosaic, I followed the algorithm described in "Multi-Image Matching using Multi-Scale Oriented Patches" by Brown, Szeliski, and Winder. At a high-level, I first detected interest points in each image, extracted an 8x8 patch around each interest point to use as a descriptor vector, matched interest points between two images based on how similar their descriptors were to each other, and computed homographies between pairs of images using RANSAC for robustness to outliers (false matches).

First, I found interest points in each image based on the multi-scale Harris corners algorithm.

Next, it's helpful to filter down to a select subset of interest points. The first reason is that the more interest points there are in each image, the more computationally expensive it is to find matches. The second reason is that when stitching images together, adjacent images may only overlap in a portion of the scene. Therefore, it is useful to choose interest points that are well distributed across the entire image to maximize the likelihood of finding true matches.

Brown et al. introduced a method called adaptive non-maximal suppression (ANMS) that keeps an interest point only if it is the strongest interest point (as measured by its corner strength) within a certain radius.

To calculate the corner strength at a pixel (x, y), we need to compute its Harris matrix: the smoothed outer product of the image gradient at (x, y). Conceptually, the Harris matrix represents a weighted average of the image's gradients near (x, y). In the equation below, ∇_{σ_d}P is the gradient of the image smoothed by convolving it with a Gaussian kernel parameterized by sigma_d.

$$\mathbf{H}_l(x,y) = \left((\nabla_{\sigma_d} P_l)^\mathsf{T} \quad \nabla_{\sigma_d} P_l \right) * g_{\sigma_i}(x,y)$$

The corner strength of an interest point (x, y) is proportional to the determinant of the Harris matrix divided by the trace of the Harris matrix.

$$f_{HM}(x,y) = \frac{\det H(x,y)}{\operatorname{tr} H(x,y)}$$

Now that we know how to calculate the corner strength for each interest point, we are ready to dive into the ANMS algorithm! n_ip represents the desired number of interest points we want to keep.

1. To implement the ANMS algorithm, we first calculate the suppression radius r_i for each interest point. The suppression radius measures how much further we must search around a given point p_i before we find a meaningfully stronger point p_j (a point whose corner strength is larger than p_i's by a ratio c_robust).

$$r_i = \min_j |p_i - p_j|, \quad s.t.\ f_{HM}(p_i) < c_{robust} * f_{HM}(p_j)$$

a. For each interest point p_i, find all other points p_j with a stronger corner strength.

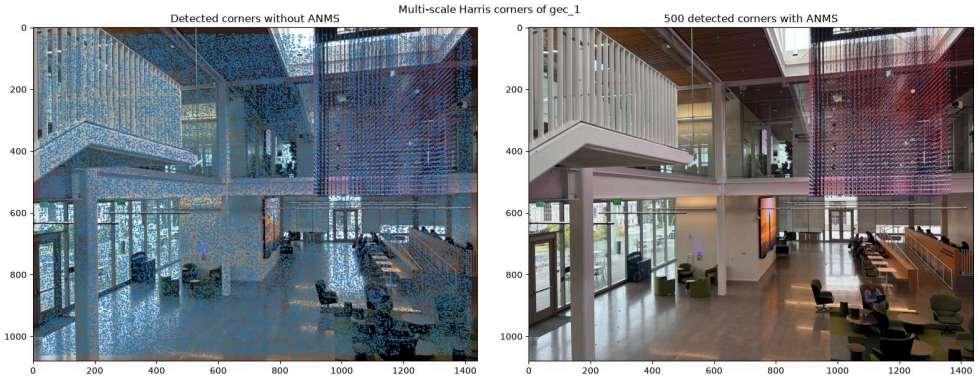
b. Then, take the distance from p_i to its nearest stronger neighbor.

c. If no other points are stronger than p_i, assign r_i = inf.

2. Sort all the interest points from largest suppression radius to smallest suppression radius.

3. Keep the n_ip interest points with the largest suppression radius.

Here are a couple images showing the detected Harris corners without ANMS and with ANMS.



Detected interest points of the Grimes Engineering Center, without adaptive non-maximal suppression and with adaptive non-maximal suppression.

Part B.2: Feature Descriptor Extraction

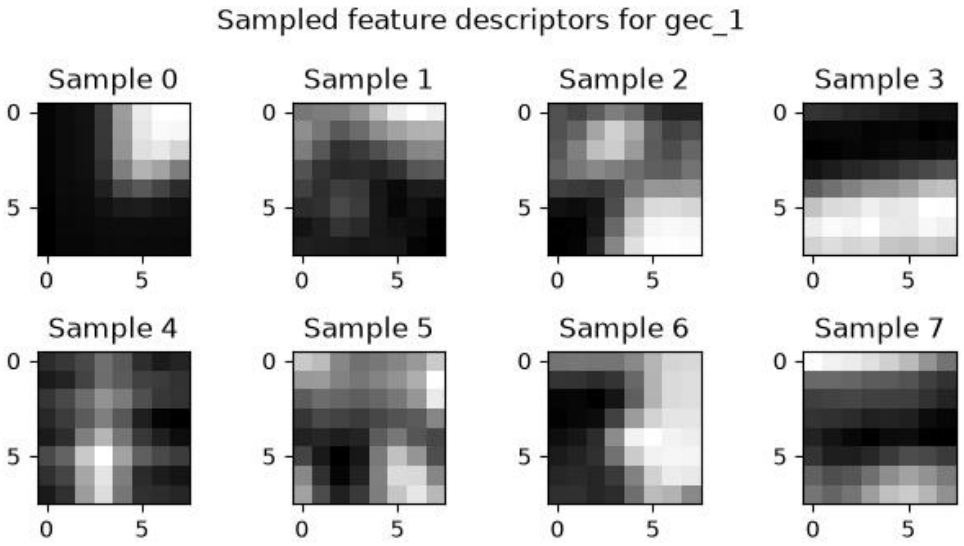
The next step is to extract feature descriptors for each interest point. The feature descriptors will help us identify which interest points in different images correspond to each other/represent the same thing in the real scene.

For our case, we'll sample a patch of 8x8 pixels around each interest point as its feature descriptor and flatten it into a vector. The 8x8 pixels were sampled such that each pixel is spaced 5 pixels apart from one another – in practice, this is like taking a 40x40 window centered around the interest point and extracting every 5th pixel. By sampling the pixels at a lowwer frequncy, this makes the patches less sensitive to the exact interest point location.

After sampling, the descriptor patch is normalized to have mean 0 and standard deviation 1 – this makes the features invariant to any affine changes in intensity (AKA bias and gain invariant).

I simplified my algorithm by omitting the rotation-invariance step and wavelet transformation step from Brown et al.'s paper.

Here are a few of the normalized 8x8 feature descriptors extracted from the picture of the Grimes Engineering Center.



A few of the normalized 8x8 feature descriptor patches from a picture of the Grimes Engineering Center.

Part B.3: Feature Matching

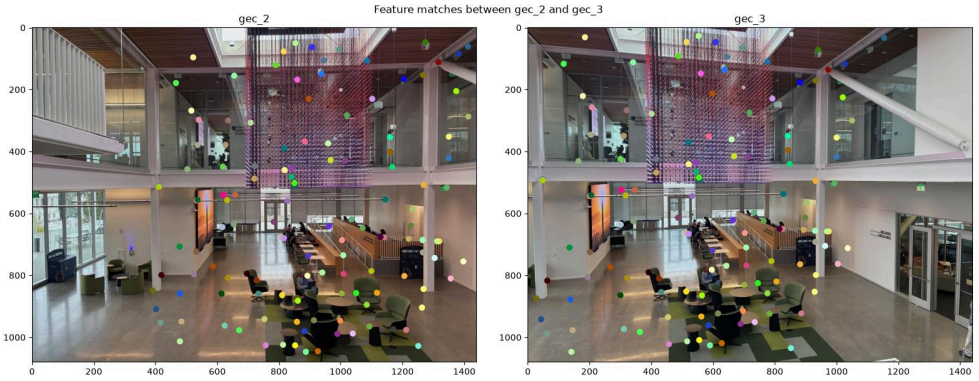
Feature matching involves finding pairs of interest points from two different images that look similar and are likely to represent the same thing in the real-world scene.

Given a list of feature descriptors extracted from one image and a list of feature descriptors extracted from aother image, I first computed the squared L2 distance between each possible pair of feature descriptors. Then, I iterated through the first list of feature descriptors. For each feature descriptor from image 1, I took the error of the best match (i.e. the distance to its first nearest neighbor) and the error of the second best match (i.e. the distance to its second nearest neighbor). Based on work by Lowe, I only accepted the first nearest neighbor as a match if it was significantly closer than the next closest neighbor. This is equivalent to accepting the first nearest neighbor only if dist(1-NN) / dist(2-NN) <= some set threshold. Likewise, I iterated through each feature descriptor

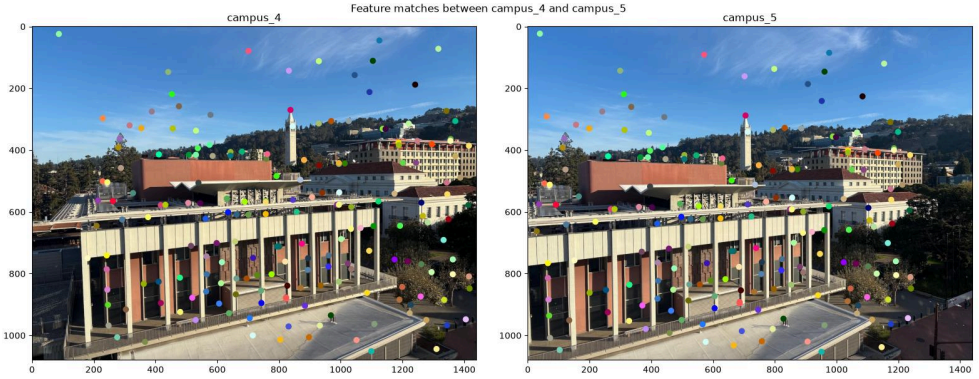
from image 2 and also applied the Lowe ratio test. Finally, I only kept symmetric matches: when feature descriptor j from image 2 was feature descriptor i from image 1's closest match AND when feature descriptor i from image 1 was feature descriptor j from image 2's closest match.

In my algorithm, I used a Lowe ratio threshold of 0.4 because based on figure 6b a threshold of 0.4 increases the number of correctly accepted matches without allowing many incorrect matches.

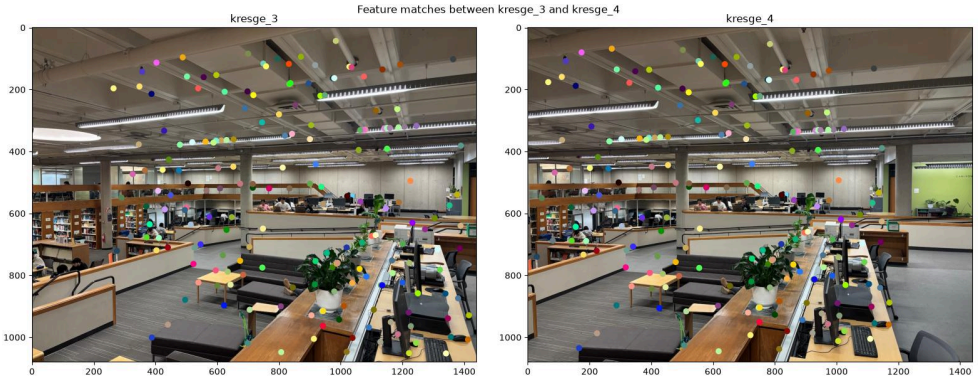
These are the matching features that my automatic feature-matching algorithm found between different pairs of images! Matching features share the same color marker in the plots.



Feature matches between image 2 and image 3 of the Grimes Engineering Center.



Feature matches between image 4 and image 5 of the Berkeley campus.



Feature matches between image 3 and image 4 of Kresge Library.

Part B.4: RANSAC for Robust Homography

Lastly, I used RANSAC (Random Sample Consensus) to estimate the homography transformation between overlapping images. RANSAC is an iterative algorithm used to estimate the best-fitting parameters for a dataset that contains a high degree of noise or has a high proportion of outliers. Naively calculating a best-fit homography matrix H using all the feature matches might inadvertantly try to also fit to incorrect correspondence pairs (outliers). RANSAC computes more robust homographies by finding an H matrix that maximizes the number of inliers (correspondence pairs that get accurately mapped to one another by a given candidate H matrix) and ignores outliers.

Here's how the 4-point RANSAC algorithm works:

1. Select 4 feature pairs at random.

2. Compute the homography H on the sampled feature pairs by solving a system of linear equations.

3. Apply the candidate H to transform im1 to im2, then compute the number of inliers (how many points from im1 after transforming by H lie within a chosen threshold from the true matching point in im2).

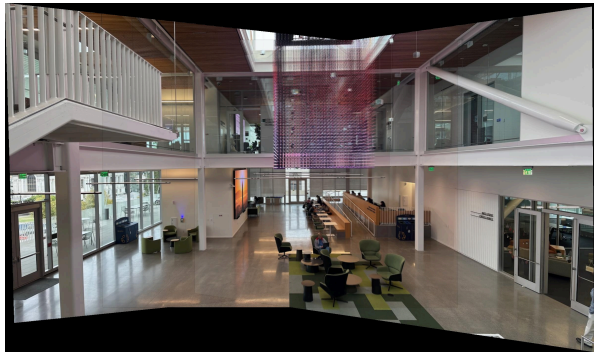
4. Repeat stpes 1-3, and keep the largest set of inliers.

5. Re-compute a least-squares H estimate using all the correspondences from the best set of inliers.

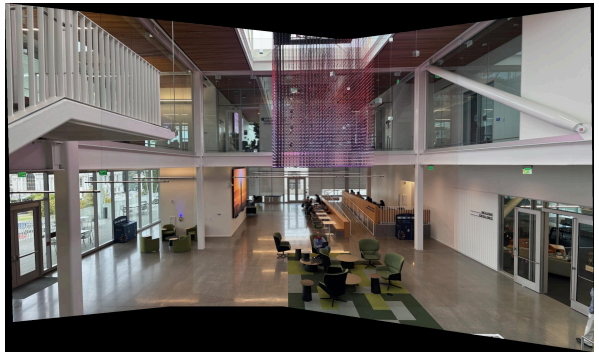
Putting all the pieces together, we now have an automatic image stitching algorithm for creating mosaics! Here are the images I manually stitched together in part A by hand-selecting feature matches and the same images automatically stitched together.

Automatic stitching improves alignment of adjacent images in the mosaic. For example, the top right corner of the Strauch Hypercube in the Grimes Engineering Center mosaic; the trees, Student Union's roof, and campanile in the Berkeley campus mosaic; and details on the plants, keyboard, and ceiling lights in the Kresge Library mosaic show more ghosting/blurring in the manually-stitched version. Automatic stitching can find way more feature matches and at much higher pixel accuracies than I can by hand :)

Although there were many moving parts to the mosaic-making and automatic stitching process, they all came together into a magical mosaic of computer vision concepts!



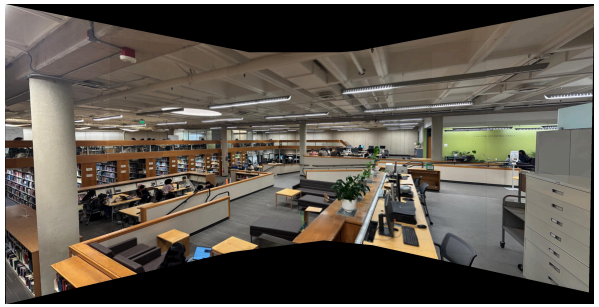
A manually-stitched photo mosaic of the Grimes Engineering Center.



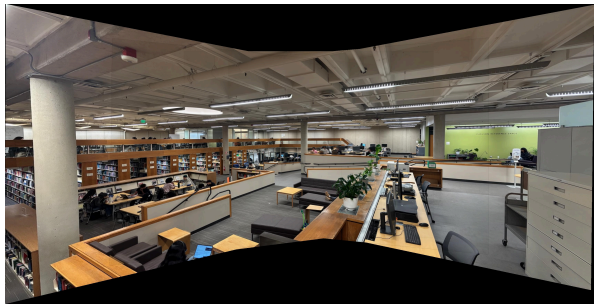
The same images of the Grimes Engineering Center, automatically stitched together into a photo mosaic.

A manually-stitched photo mosaic of the Berkeley campus.

The same images of the Berkeley campus, automatically stitched together into a photo mosaic.



A manually-stitched photo mosaic of the Kresge Library.



The same images of the Kresge Library, automatically stitched together into a photo mosaic.